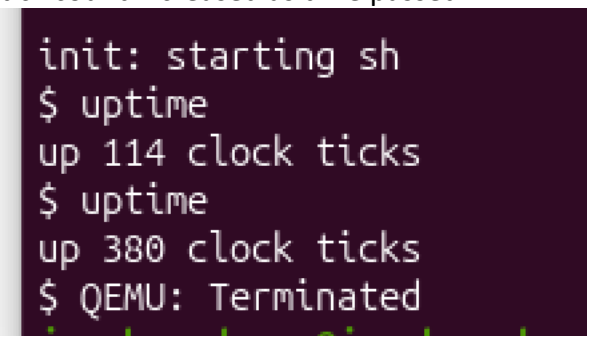
CS4475 Fall 2026 Lab Report
Jacob Ordonez 09/23/2026
Jjordonez@miners.utep.edu

PA 1: Implementing xv6 User Commands and System Calls

Please replace red text with your report text and any tables or figures, names of any accompanying files, etc. Remember to commit all the files for your lab submission, to put the URL for your private xv6 repo in the Teams assignment, to submit the Teams assignment, and to give the instructor and TA access to your repo.

Task 1. (5 pts) Implement the uptime utility

I implemented the uptime command in user/uptime.c on the pa1-syscalls branch. The program calls the uptime() system call to get the number of clock ticks since xv6 started, prints the result, and exits. I also added the command to UPROGS in the Makefile so it would be included in xv6. I ran the command twice, and it printed 114 ticks and then 380 ticks, showing that the tick count increased as time passed.



Task 2. (10 pts) Implement a time1 command that reports elapsed time.

I implemented time1 in user/time1.c and added it to UPROGS in the Makefile. The program records the starting tick count using uptime(), creates a child with fork(), and uses exec() to run the requested command with its arguments. The parent calls wait() and then subtracts the starting tick count from the ending count to calculate elapsed time. It also handles missing arguments and failures from fork(), exec(), and wait().

During testing, time1 sleep 10 reported 10 elapsed ticks. A single time1 matmul run reported 56 elapsed ticks. When two copies ran concurrently on one CPU, they reported 117 and 116 elapsed ticks because they shared CPU time. Both returned the expected matrix result of 6000. I changed the matrices in matmul.c to static arrays to avoid exceeding the stack and added a result check.

```
$ time1 matmul &; time1 matmul &
$ time1 matmul &; time1 matmul &Result check: 6000
Time: 114 ticks
Result check: 6000
Time: 111 ticks
Elapsed time: 117 ticks
Elapsed time: 116 ticks
```

Task 3. (10 pts) Keep track of how much cputime a process has used.

I added CPU-time tracking by modifying three kernel files. In kernel/proc.h, I added a uint field named cputime to struct proc to store each process's CPU tick count. In kernel/proc.c, I initialized this field to zero in allocproc() so every new process starts with zero CPU ticks. In kernel/trap.c, I incremented the counter before yield() when a timer interrupt occurs in usertrap() or kerneltrap(). This counts ticks while the process runs in either user mode or kernel mode.
After rebuilding xv6, I ran time1 matmul. The program returned the expected result of 6000 and reported 51 elapsed ticks. This confirmed that the modified kernel could boot and run the program. The CPU-time counter will be checked after implementing wait2 and the time utility.

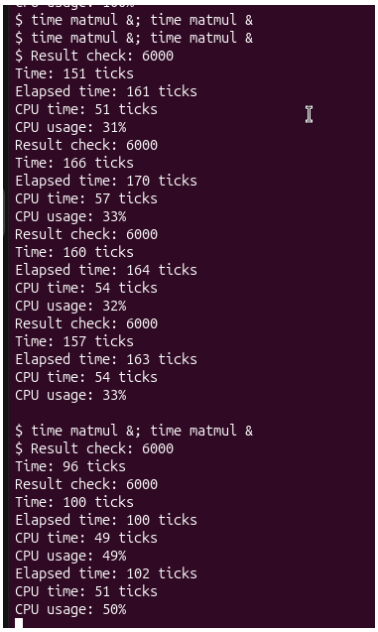
```
xv6 kernel is booting

init: starting sh
$ time1 matmul
Result check: 6000
Time: 48 ticks
Elapsed time: 51 ticks
$
```

Task 4. (10 pts) Implement a wait2() system call that waits for a child to exit and returns the child's status and rusage.
I implemented wait2 to return a child process's exit status and CPU usage. I created kernel/pstat.h with struct rusage, containing the cputime field. In user/user.h, I declared the structure and wait2 function. I added the syscall entry in user/usys.pl, assigned syscall number 23 in kernel/syscall.h, and registered its handler in kernel/syscall.c.
In kernel/sysproc.c, I added sys_wait2 to retrieve the two user pointers and call kwait2. I declared kwait2 in kernel/defs.h and implemented it in kernel/proc.c using the existing kwait logic. When a child finishes, kwait2 uses copyout() to return its exit status and CPU usage before freeing the child's process structure. It returns the child's PID on success or -1 on failure.

The modified kernel compiled and booted successfully. Functional testing will follow using the time utility.

Task 5. (10 pts) Implement a time command that runs the command given to it as an argument and outputs elapsed time, CPU time, and %CPU used.
I implemented the time utility using fork() and exec() to run a command and wait2() to obtain its exit status and CPU time. Elapsed time is measured using uptime(). CPU usage is calculated as (CPU time × 100) / elapsed time, with a check to avoid division by zero.
Running time sleep 10 produced 10 elapsed ticks and 0 CPU ticks, giving 0% CPU usage because the process spent its time sleeping. Running time matmul alone produced 51 elapsed ticks and 51 CPU ticks, giving 100% CPU usage. When two matrix computations ran concurrently, one reported 100 elapsed ticks and 49 CPU ticks (49%), while the other reported 102 elapsed ticks and 51 CPU ticks (50%). These results show that the two computations shared the single CPU. Both returned the expected result of 6000.



Task 6. (5 pts) Summarize what you learned from doing this assignment. Describe any difficulties you faced and if/how you overcame them.
Through this assignment, I learned how xv6 user programs interact with the kernel through system calls. I practiced using `fork()`, `exec()`, and `wait()` to run a command and wait for it to finish. Adding a CPU tick counter and the `wait2()` system call helped me understand how the kernel tracks information about each process and returns it to a user program.

I also learned the difference between elapsed time and CPU time. The sleep test showed that a process can take time to finish while using almost no CPU time. The matrix multiplication test

used approximately 100% of the CPU when running alone. When two computations ran together on one CPU, each used approximately 50%, showing how processes share the processor.

One difficulty was distinguishing the xv6 shell from the Ubuntu terminal. Initially, I tried running Git commands inside xv6. I resolved this by learning how to exit QEMU and return to Ubuntu. I also had trouble running two commands concurrently because the xv6 shell required different syntax, and I accidentally launched extra processes by entering the command twice. Using the correct syntax and waiting for both programs to finish gave me a clear test result. Working through these issues taught me to make small changes, check the output carefully, and test each part before moving on.

Extra Credit (5 points). Discuss limitations of our time command. (Hint: For one limitation, consider what would happen if the command that is being timed forks child processes).

Our time command reports CPU time for the process it starts directly. If that process forks children, wait2() returns only the direct child's CPU time, so the reported CPU time and CPU usage leave out work done by its children. If the direct child waits for them, their work increases elapsed time without increasing the reported CPU time. If it exits while they are still running, time may finish reporting before they do.
The measurements also have limited precision because xv6 counts whole clock ticks. A short command may finish between timer interrupts and report zero CPU ticks or zero elapsed ticks, even though it did run.

I used ChatGPT as a programming assistant during this assignment. It helped explain xv6 system-call structure, suggest implementation steps, troubleshoot compiler and QEMU issues, and review test results. I wrote, entered, tested, and modified the code in my own xv6 repository and verified the behavior through the required tests. I understand the implementation and can explain it during the demonstration.